

SQL i nie tylko

Wykład trzeci

dr inż. Marcin Ciura, Uniwersytet Komisji Edukacji Narodowej

Proszę zrobić
notatki :-)

Plan na dziś

Plan na dziś

Z tego wykładu dowiedziecie się:

- + o klauzuli WHERE

- + o tym, jak zmieniać bazę danych instrukcją
ALTER TABLE i UPDATE

- + o tym, jak zadawać zapytania
ze złączeniami, czyli z klauzulą JOIN

Przepraszam was za to, że dziś
na slajdach nie będzie zdjęć :-)

Instrukcja ALTER TABLE

Instrukcja ALTER TABLE

W SQL-u instrukcja ALTER TABLE:

- + albo zmienia kolumny istniejącej tabeli
- + albo zmienia nazwę tabeli

Instrukcja ALTER TABLE

A. Zmiana nazwy tabeli (RENAME TO...)

```
ALTER TABLE Gatunek RENAME TO GatunkiDrzew;
```

System zarządzania bazą danych zmienia też deklaracje kluczy obcych i wyzwalaczy związanych z kluczem głównym tabeli

Instrukcja ALTER TABLE

B. Zmiana nazwy kolumny (RENAME COLUMN TO...)

```
ALTER TABLE GatunkiDrzew
```

```
RENAME COLUMN nazwa TO nazwa_gatunku;
```

System zarządzania bazą danych zmienia nazwę kolumny także w indeksach, wyzwalaczach i widokach

Instrukcja ALTER TABLE

C. Dodanie kolumny (ADD COLUMN...)

```
ALTER TABLE Pracownik  
ADD COLUMN ulubiona_zupa;
```

```
ALTER TABLE PracownikZespołu  
ADD COLUMN wypłata_EUR;
```

System zarządzania bazą danych dopisuje daną kolumnę jako ostatnią kolumnę tabeli

Instrukcja ALTER TABLE

D. Usunięcie kolumny (DROP COLUMN...)

```
ALTER TABLE GatunkiDrzew  
DROP COLUMN nazwa_łacińska;
```

System zarządzania bazą danych usuwa kolumnę

Instrukcja ALTER TABLE

E. Ograniczenia SQLite

W SQLite, inaczej niż w standardzie SQL, nie da się za pomocą instrukcji ALTER TABLE:

- + zmienić typu kolumny
- + dodać ani usunąć klucza głównego
- + dodać klucza obcego

Klauzula WHERE

Klauzula WHERE

Klauzula WHERE warunek działa jak sito

Przepuszcza tylko te wiersze, dla których dany warunek jest spełniony

Klauzula WHERE

Możemy używać klauzuli WHERE:

- + w zapytaniach SELECT
- + w instrukcjach UPDATE
- + w instrukcjach DELETE

Klauzula WHERE

A. Operatory porównania – równość:

= lub ==

(oba sposoby zapisu tego operatora działają)

Klauzula WHERE

```
SELECT imię, nazwisko  
FROM Pracownik  
WHERE imię = 'Olga';
```

imię	nazwisko
Olga	Owca

Klauzuła WHERE

```
SELECT imię, nazwisko  
FROM Pracownik  
WHERE nazwisko = 'Lis';
```

imię	nazwisko
Lucyna	Lis

Klauzula WHERE

B. Operatory porównania – nierówność:

`!=` lub `<>`

(oba sposoby zapisu tego operatora działają)

Klauzula WHERE

```
SELECT imię, nazwisko  
FROM Pracownik  
WHERE imię != 'Wiktor';
```

imię	nazwisko
Olga	Owca
Kasia	Kot
Lucyna	Lis

Klauzula WHERE

```
SELECT imię, nazwisko  
FROM Pracownik  
WHERE imię <> 'Kasia';
```

imię	nazwisko
Olga Wiktor Lucyna	Owca Wilk Lis

Klauzula WHERE

C. Porównania na mniejszość i na większość
(działają i na liczbach, i alfabetycznie
na tekstach):

<, <=, =>, >

Klauzula WHERE

```
SELECT imię, nazwisko  
FROM Pracownik  
WHERE imię <= 'Olga';
```

imię	nazwisko
Olga	Owca
Kasia	Kot
Lucyna	Lis

Klauzula WHERE

D. Bezpieczne porównania z wartością NULL:

IS NULL, IS NOT NULL

Klauzula WHERE

Prawie każde działanie z użyciem NULL daje w wyniku NULL

```
SELECT 2+2;
```

2+2
4

Klauzula WHERE

```
SELECT 2+NULL;
```

2+NULL
NULL

Klauzula WHERE

```
SELECT 2 = NULL;
```

2 = NULL
NULL

Klauzula WHERE

Wyjątek to operatory IS NULL i IS NOT NULL:

```
SELECT 2 IS NULL;
```

2 IS NULL
0

0 oznacza wartość logiczną „nieprawda”

Klauzula WHERE

Wyjątek to operatory IS NULL i IS NOT NULL:

```
SELECT 2 IS NOT NULL;
```

2 IS NOT NULL
1

1 oznacza wartość logiczną „prawda”

Klauzula WHERE

Wyjątek to operatory IS NULL i IS NOT NULL:

```
SELECT NULL IS NULL;
```

NULL IS NULL
1

Klauzula WHERE

```
SELECT id_pracownika  
FROM PracownikZespołu  
WHERE kwota_wypłaty_PLN IS NULL;
```

id_pracownika
1
2
3

kwota_wypłaty_PLN równa NULL oznacza, że pracownik dopiero dostanie tę wypłatę

Klauzula WHERE

E. łączenie warunków – operatory logiczne

Możemy tworzyć złożone warunki, łącząc prostsze warunki operatorami logicznymi

AND, OR i NOT

Wskazówka: gdy używamy w warunku zarówno AND, jak i OR, lepiej używać nawiasów

Klauzula WHERE

```
SELECT id_pracownika, kwota_wypłaty_PLN  
FROM PracownikZespołu  
WHERE kwota_wypłaty_PLN <= 12000 OR kwota_wypłaty_PLN IS NULL;
```

id_pracownika	kwota_wypłaty_PLN
2	10000.0
1	12000.0
1	NULL
2	NULL
3	NULL

Klauzula WHERE

F. Zaawansowane operatory filtrowania

Przedziały – operator BETWEEN dół AND góra

Klauzula WHERE

```
SELECT id_pracownika, kwota_wypłaty_PLN  
FROM PracownikZespołu  
WHERE kwota_wypłaty_PLN BETWEEN 12000 AND 15000;
```

id_pracownika	kwota_wypłaty_PLN
1	15000.0
1	12000.0
2	15000.0

Klauzula WHERE

G. Zaawansowane operatory filtrowania

Przynależność do zbioru – operator IN

Operator IN sprawdza, czy wartość znajduje się na liście lub w wyniku podzapytania

Klauzula WHERE

```
SELECT imię, nazwisko  
FROM Pracownik  
WHERE nazwisko IN ('Owca', 'Wilk');
```

imię	nazwisko
Ołga Wiktor	Owca Wilk

Klauzula WHERE

```
SELECT imię, nazwisko  
FROM Pracownik  
WHERE id_pracownika IN (  
    SELECT id_pracownika  
    FROM PracownikZespołu  
    WHERE kwota_wypłaty_PLN > 14000);
```

imię	nazwisko
Olga	Owca
Wiktor	Wilk
Lucyna	Lis

Klauzula WHERE

H. Zaawansowane operatory filtrowania

Dopasowanie do wzorców tekstowych – operator
LIKE

Operator LIKE sprawdza, czy wartość pasuje
do wzorca tekstowego

Klauzula WHERE

Operator **LIKE** sprawdza, czy wartość pasuje do wzorca tekstowego

Operator **LIKE** nie zwraca uwagi na wielkość liter

Operator **LIKE** korzysta z dwóch znaków wieloznacznych:

- + **_** zastępuje dokładnie jeden znak
- + **%** zastępuje 0, 1, 2, 3... znaki

Klauzula WHERE

```
SELECT imię, nazwisko  
FROM Pracownik  
WHERE imię LIKE '%a';
```

imię	nazwisko
Olga	Owca
Kasia	Kot
Lucyna	Lis

Klauzula WHERE

```
SELECT imię, nazwisko  
FROM Uczeń  
WHERE nazwisko LIKE 'Kowalsk_';
```

imię	nazwisko
Anna	Kowalska
Jan	Kowalski

Instrukcja UPDATE

Instrukcja UPDATE

Instrukcja UPDATE zmienia wartości niektórych kolumn w wierszach danej tabeli

```
UPDATE NazwaTabeli  
SET kolumna1 = wartość1, kolumna2 = wartość2  
WHERE warunek;
```

Instrukcja UPDATE

UWAGA: Proszę pamiętać o klauzuli WHERE :-)

Jeśli nie napiszemy klauzuli WHERE, system zarządzania bazą danych nie zgłosi błędu

Zamiast tego zmieni wszystkie wiersze tabeli

```
UPDATE PracownikZespołu -- podwyżka  
SET kwota_wypłaty_PLN = 1.2*kwota_wypłaty_PLN;
```

Instrukcja UPDATE

Na końcu ostatniego wykładu zostawiliśmy naszych bohaterów bez przydziału do działów:

```
SELECT * FROM Pracownik;
```

id_pracownika	id_działu	imię	nazwisko
1	NULL	Olga	Owca
2	NULL	Wiktor	Wilk
3	NULL	Kasia	Kot
4	NULL	Lucyna	Lis

Instrukcja UPDATE

Naprawmy ten błąd:

```
UPDATE Pracownik
SET id_działu = 1  -- Dział Analiz
WHERE imię = 'Kasia' OR imię = 'Lucyna';
SELECT * FROM Pracownik;
```

id_pracownika	id_działu	imię	nazwisko
1	NULL	Olga	Owca
2	NULL	Wiktor	Wilk
3	1	Kasia	Kot
4	1	Lucyna	Lis

Instrukcja UPDATE

Możemy też użyć operatora **IN**:

```
UPDATE Pracownik  
SET id_działu = 2 -- Dział Wdrożeń  
WHERE imię IN ('Olga', 'Wiktor');  
SELECT * FROM Pracownik;
```

id_pracownika	id_działu	imię	nazwisko
1	2	Olga	Owca
2	2	Wiktor	Wilk
3	1	Kasia	Kot
4	1	Lucyna	Lis

Proszę o pytania :-)

Złączenia

Złączenia

Znamy już zapytania do jednej tabeli
bez klauzuli WHERE i z klauzulą WHERE:

```
SELECT kolumna1, kolumna2  
FROM Tabela;
```

```
SELECT kolumna1, kolumna2  
FROM Tabela  
WHERE warunek;
```

Złączenia

Czas poznać zapytania do wielu tabel
z użyciem złączeń

Złączenia

Tak zapisujemy złączenia:

```
SELECT ... FROM Tabela1 JOIN Tabela2  
ON Tabela1.kolumna1 = Tabela2.kolumna2;
```

Zamiast = można by też użyć w złączeniu innego operatora, np. >, <, <>. Rzadko jest to potrzebne.

Złączenia

```
SELECT ... FROM Tabela1 JOIN Tabela2  
USING (kolumna)
```

to uproszczony zapis. Stosujemy go, gdy kolumny łączące mają tę samą nazwę w obu tabelach

Złączenia

```
SELECT ... FROM Tabela1 JOIN Tabela2  
USING (kolumna1, kolumna2)
```

Proszę zwrócić uwagę na nawiasy wokół nazw kolumn łączących. Wewnątrz tych nawiasów może być kilka nazw kolumn, rozdzielonych przecinkami, zwłaszcza gdy w tabelach są **klucze złożone**, czyli takie klucze główne, które składają się z wielu kolumn

Złączenia

Klauzula **AS** tworzy aliasy tabel, czyli inne nazwy tabel

Dzięki klauzuli **AS** można skracać zapytania i czynić je zgodnymi z zasadą **DRY**

```
SELECT ...  
FROM Tabela1 AS t1 JOIN Tabela2 AS t2  
ON t1.kolumna1 = t2.kolumna2;
```

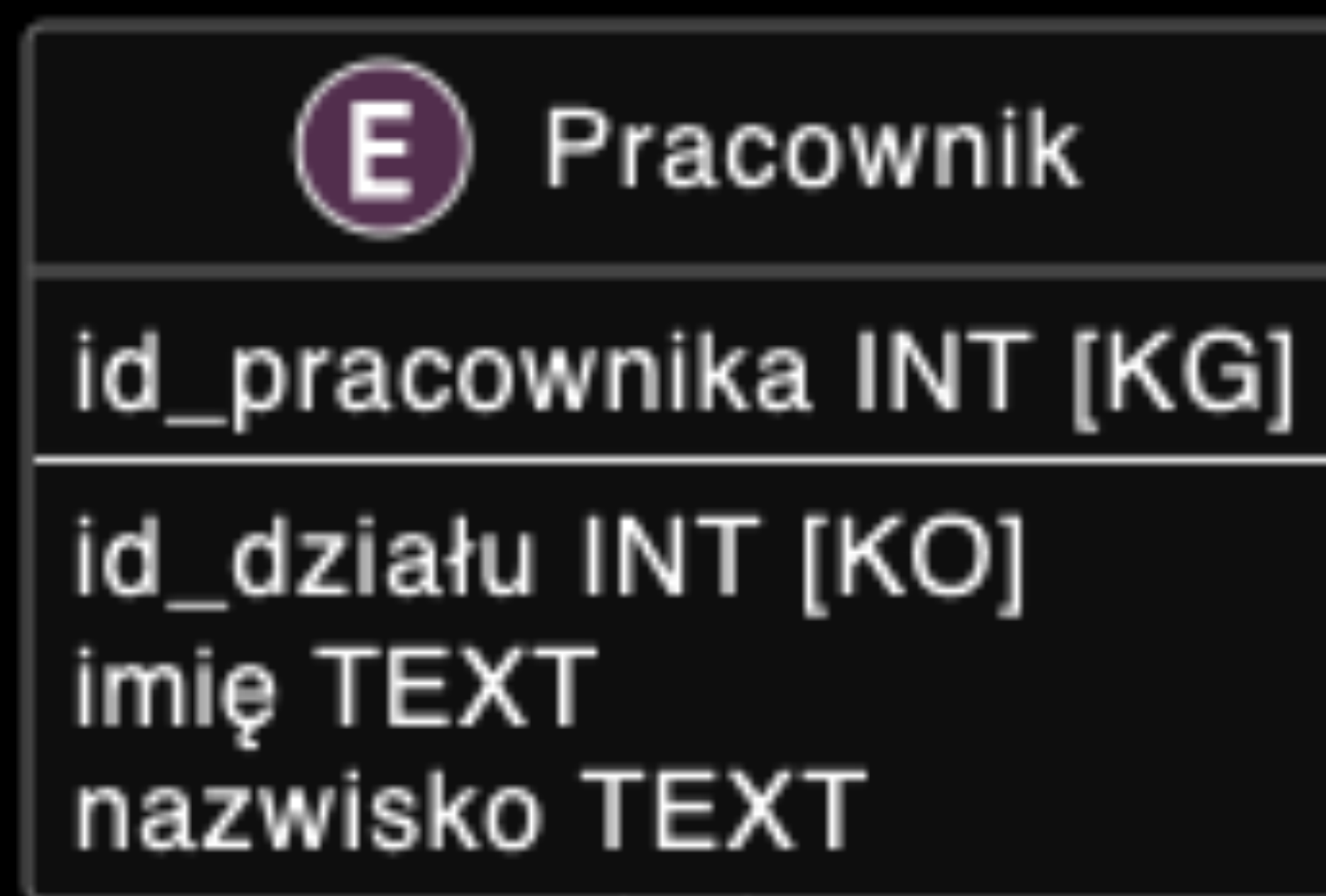
Złączenia

Możemy też opuścić wyraz **AS**, a alias tabeli i tak powstanie :-)

```
SELECT ...  
FROM Tabela1 t1 JOIN Tabela2 t2  
ON t1.kolumna1 = t2.kolumna2;
```

Złączenia

Przykład 1: Wyszukać imiona i nazwiska pracowników wraz z nazwami tych działów, do których należą



Złączenia

```
SELECT imię, nazwisko, nazwa  
FROM Pracownik  
JOIN Dział ON Pracownik.id_działu = Dział.id_działu;
```

imię	nazwisko	nazwa
Olga	Owca	Dział Wdrożeń
Wiktor	Wilk	Dział Wdrożeń
Kasia	Kot	Dział Analiz
Lucyna	Lis	Dział Analiz

Złączenia

```
SELECT imię, nazwisko, nazwa  
FROM Pracownik AS p  
JOIN Dział AS d ON p.id_działu = d.id_działu;
```

imię	nazwisko	nazwa
Olga	Owca	Dział Wdrożeń
Wiktor	Wilk	Dział Wdrożeń
Kasia	Kot	Dział Analiz
Lucyna	Lis	Dział Analiz

Złączenia

```
SELECT imię, nazwisko, nazwa  
FROM Pracownik p  
JOIN Dział d ON p.id_działu = d.id_działu;
```

imię	nazwisko	nazwa
Olga	Owca	Dział Wdrożeń
Wiktor	Wilk	Dział Wdrożeń
Kasia	Kot	Dział Analiz
Lucyna	Lis	Dział Analiz

Złączenia

```
SELECT imię, nazwisko, nazwa  
FROM Pracownik JOIN Dział USING (nr_działu);
```

imię	nazwisko	nazwa
Olga	Owca	Dział Wdrożeń
Wiktor	Wilk	Dział Wdrożeń
Kasia	Kot	Dział Analiz
Lucyna	Lis	Dział Analiz

Złączenia

Przykład 2: Wyszukać imiona i nazwiska szefów działów wraz z nazwami tych działów, którymi kierują

E Dział
id_działu INT [KG]
id_pracownika_szefa_działu INT [KO] nazwa TEXT

szef działu

E Pracownik
id_pracownika INT [KG]
id_działu INT [KO] imię TEXT nazwisko TEXT

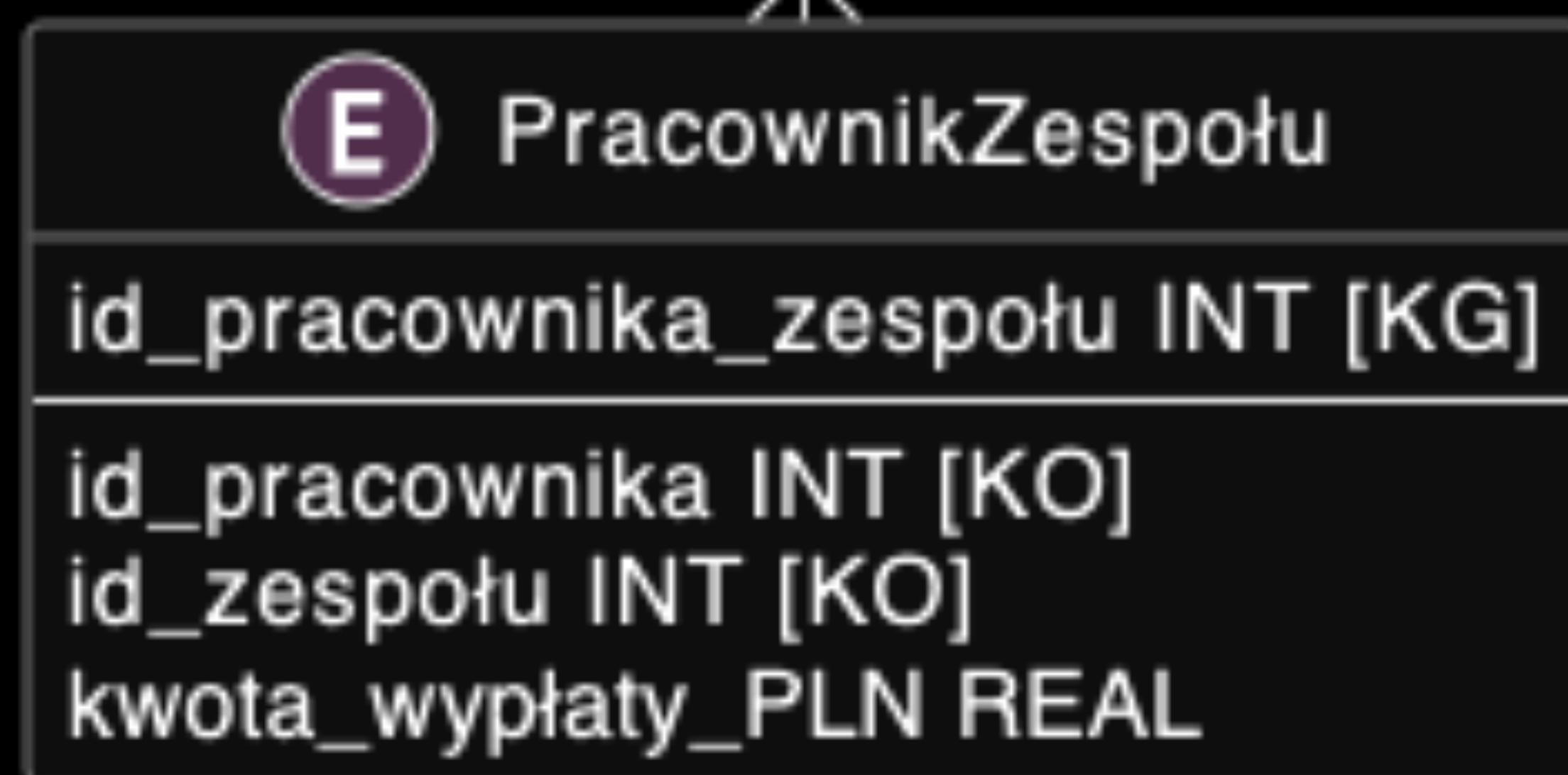
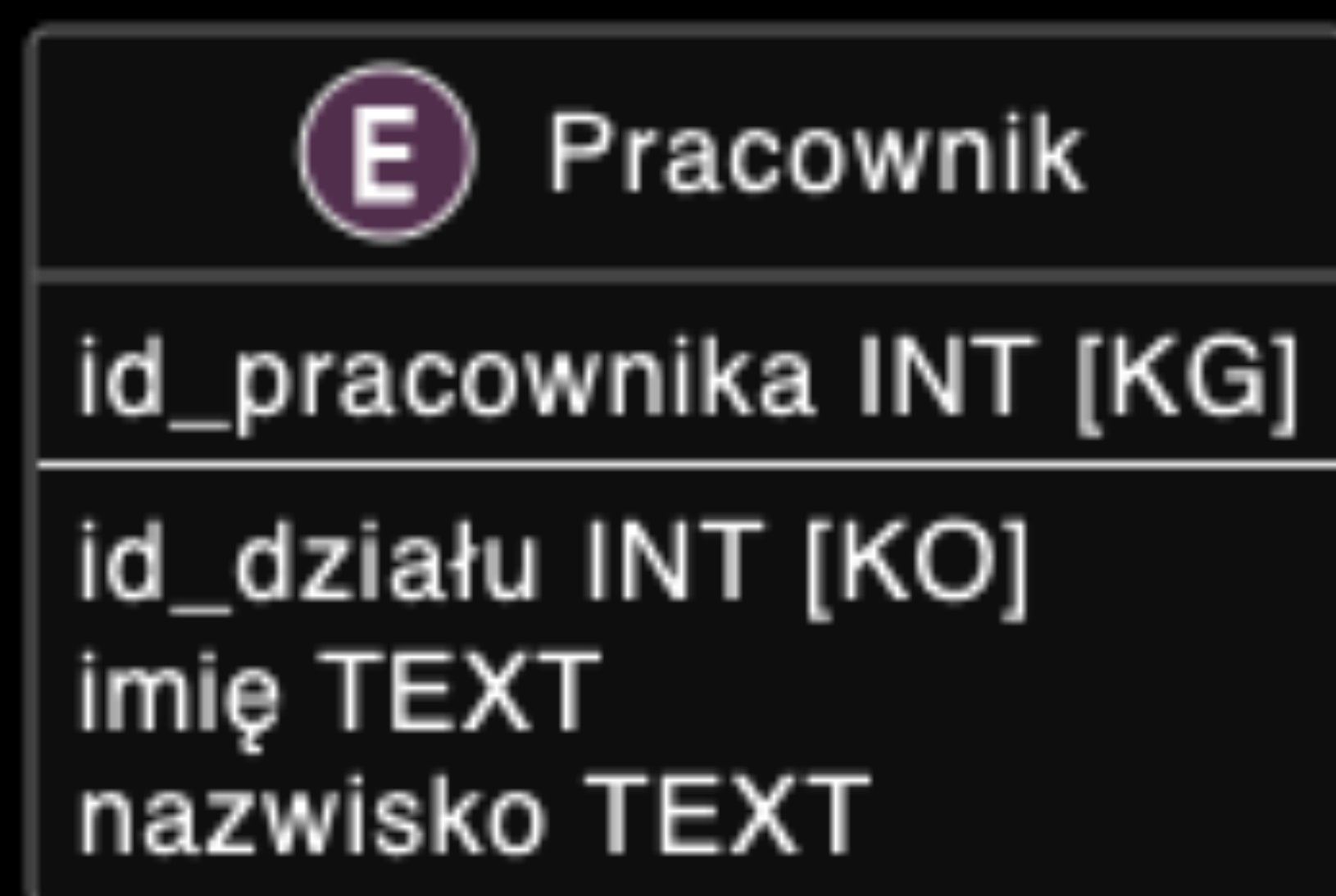
Złączenia

```
SELECT imię, nazwisko, nazwa  
FROM Pracownik p JOIN Dział d  
ON p.id_pracownika = d.id_pracownika_szefa_działu;
```

imię	nazwisko	nazwa
Kasia	Kot	Dział Analiz
Olga	Owca	Dział Wdrożeń

Złączenia

Przykład 3: Wyszukać imiona i nazwiska pracowników wraz z kwotami wypłat i numerami zespołów, w ramach których były te wypłaty



Złączenia

```
SELECT imię, nazwisko, kwota_wypłaty_PLN, id_zespołu
FROM Pracownik
JOIN PracownikZespołu USING (id_pracownika);
```

imię	nazwisko	kwota_wypłaty_PLN	id_zespołu
Wiktor	Wilk	10000.0	1
Olga	Owca	15000.0	1
Olga	Owca	12000.0	2
Wiktor	Wilk	15000.0	2
Lucyna	Lis	20000.0	2
Olga	Owca	NULL	3
Wiktor	Wilk	NULL	3
Kasia	Kot	NULL	3

Złączenia

Przykład 4: Wyszukać imiona i nazwiska pracowników wraz z kwotami wypłat i nazwami zadań, za które były te wypłaty

E Pracownik

id_pracownika INT [KG]

id_działu INT [KO]

imię TEXT

nazwisko TEXT

E Zespół

id_zespołu INT [KG]

id_pracownika_kierownika_zespołu [KO]

nazwa_zadania TEXT

E PracownikZespołu

id_pracownika_zespołu INT [KG]

id_pracownika INT [KO]

id_zespołu INT [KO]

kwota_wypłaty_PLN REAL

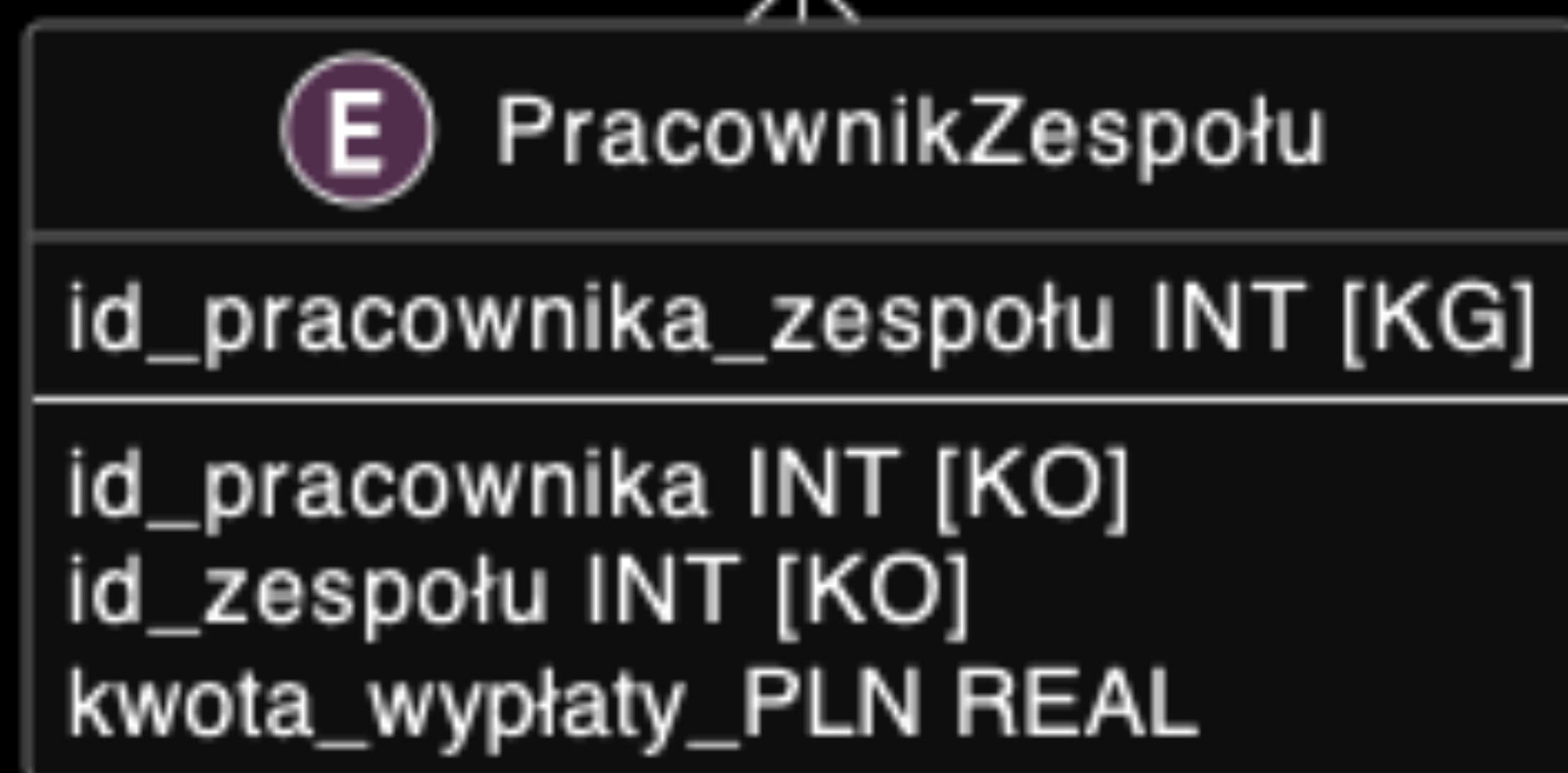
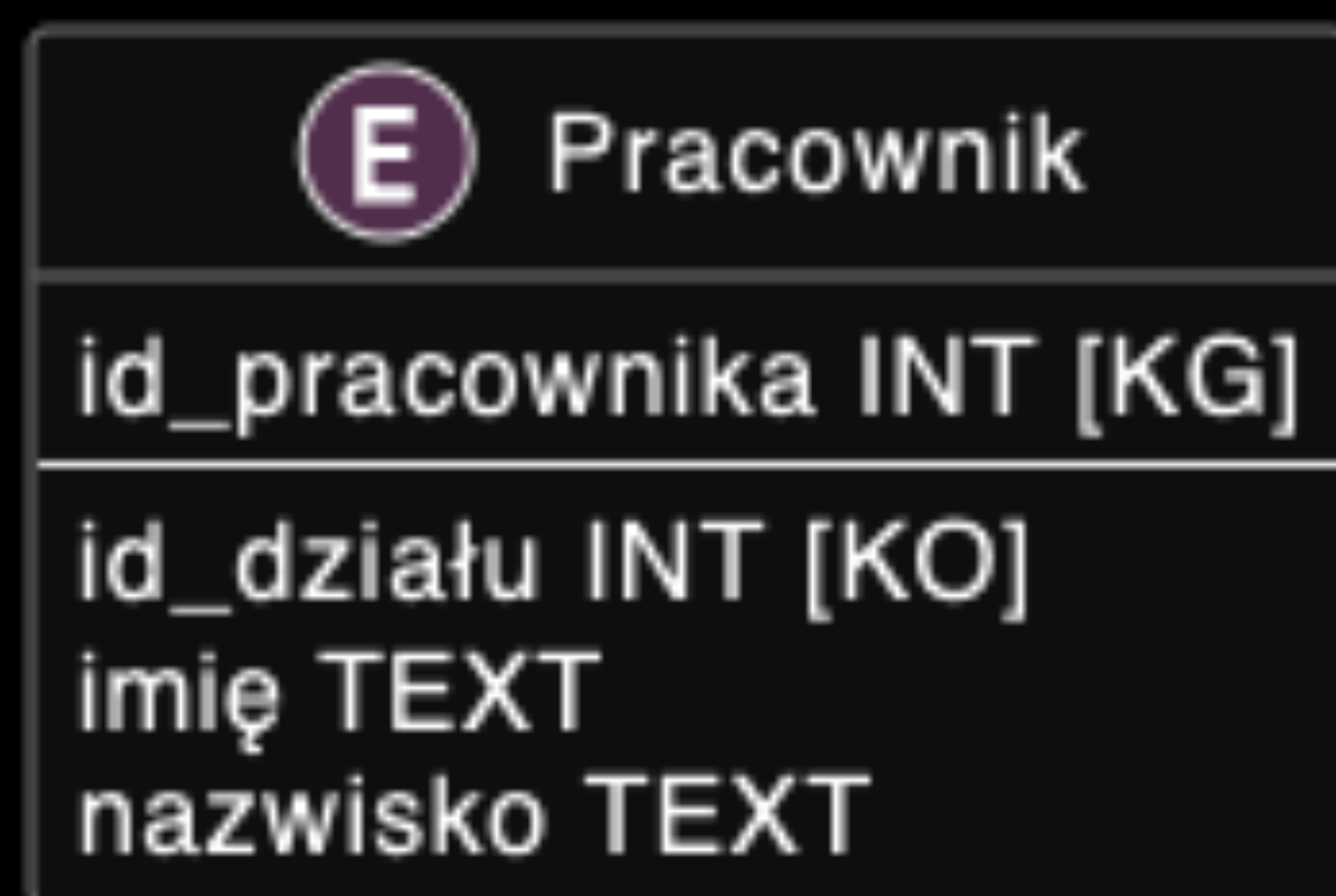
Złączenia

```
SELECT imię, nazwisko, kwota_wypłaty_PLN, nazwa_zadania
FROM Pracownik
JOIN PracownikZespołu USING (id_pracownika)
JOIN Zespół USING (id_zespołu);
```

imię	nazwisko	kwota_wypłaty_PLN	nazwa_zadania
Wiktor	Wilc	10000.0	Projekt sterownika
Olga	Owca	15000.0	Projekt sterownika
Olga	Owca	12000.0	Projekt symulatora
Wiktor	Wilc	15000.0	Projekt symulatora
Lucyna	Lis	20000.0	Projekt symulatora
Olga	Owca	NULL	Projekt konwertera
Wiktor	Wilc	NULL	Projekt konwertera
Kasia	Kot	NULL	Projekt konwertera

Złączenia

Przykład 5: Wyszukać imiona i nazwiska pracowników zespołu nr 2 wraz z kwotami wypłat (dla wypłat większych niż 13000 zł)



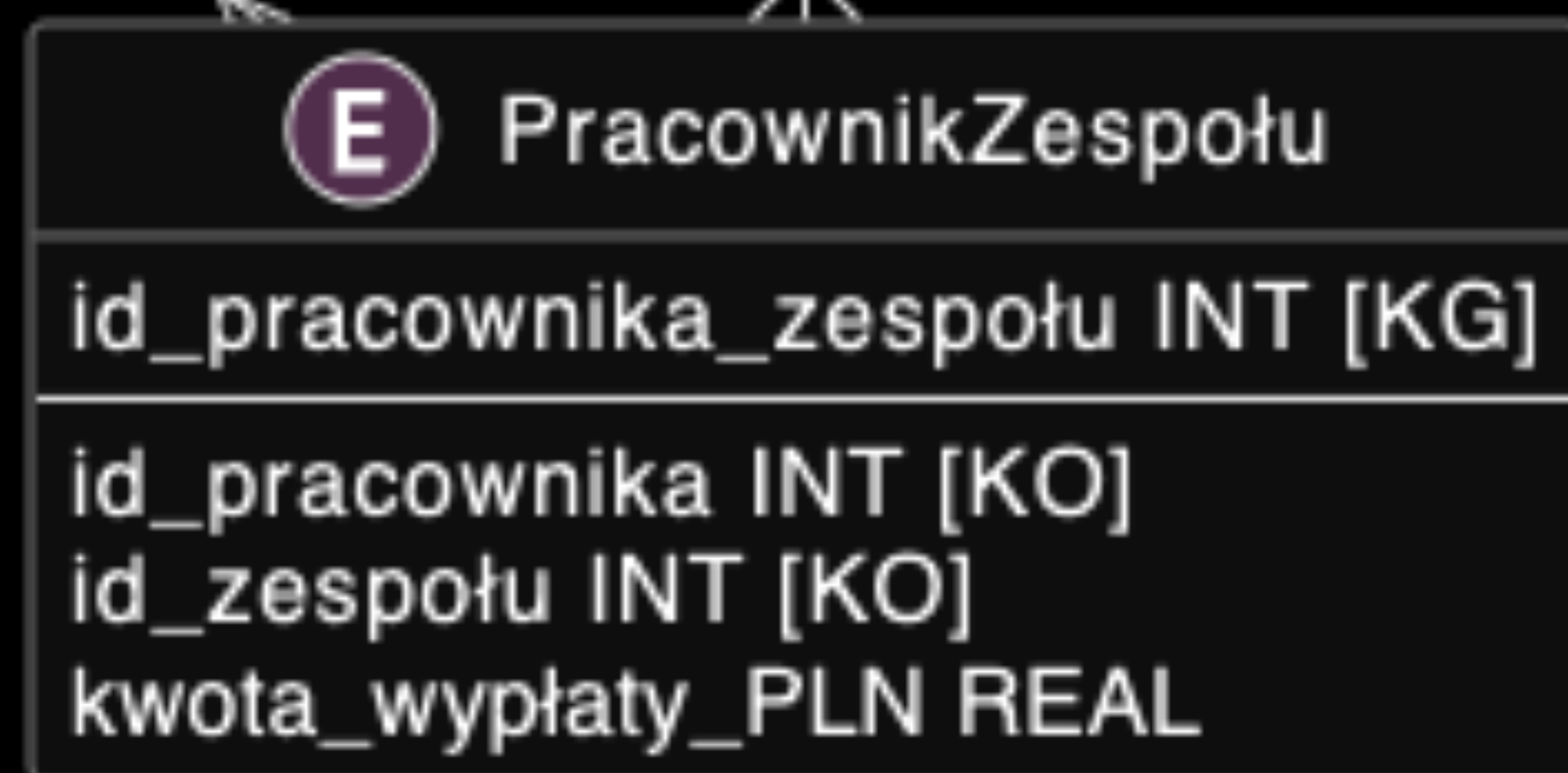
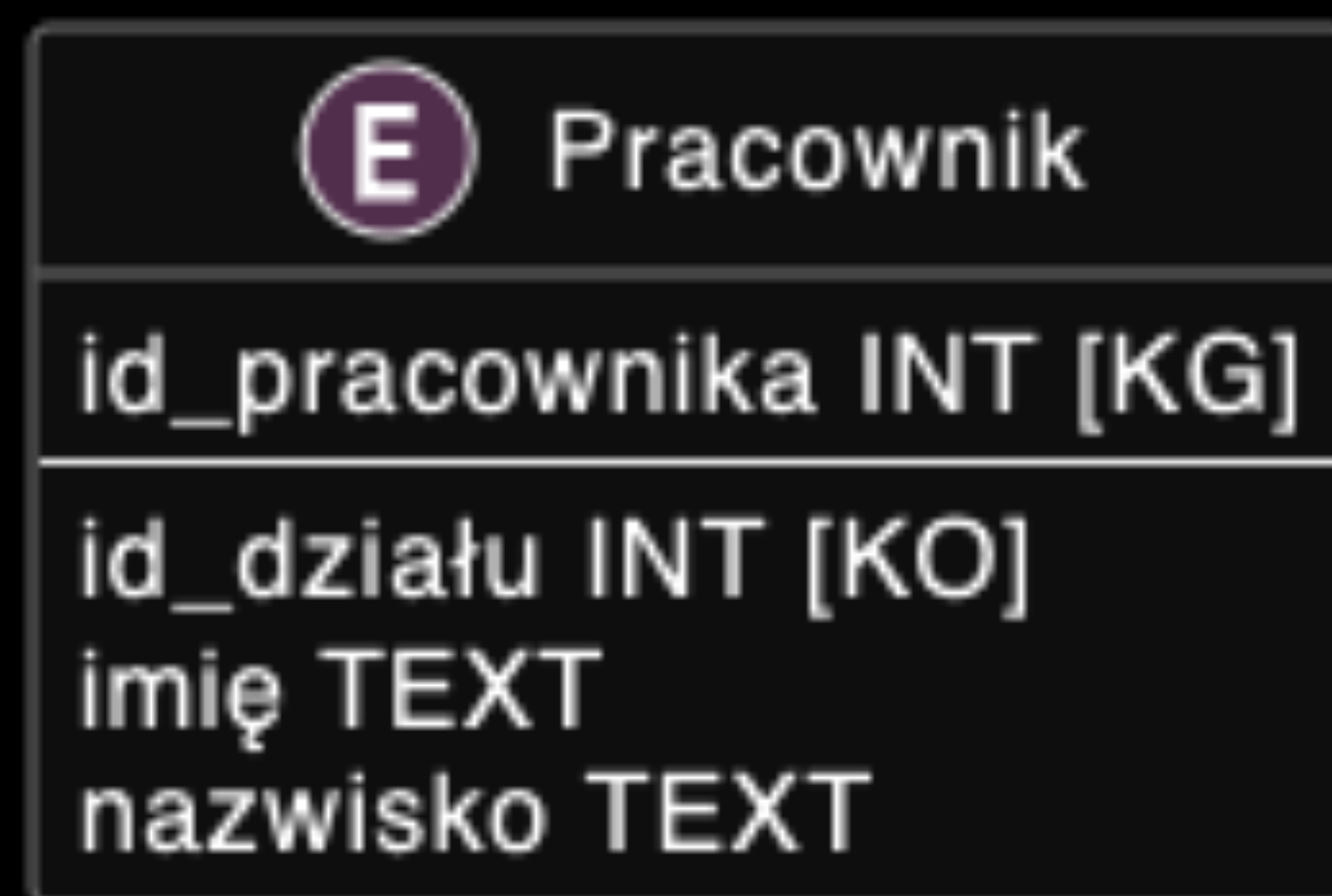
Złączenia

```
SELECT imię, nazwisko, kwota_wypłaty_PLN  
FROM Pracownik  
JOIN PracownikZespołu USING(id_pracownika)  
WHERE id_zespołu = 2  
AND kwota_wypłaty_PLN > 13000;
```

imię	nazwisko	kwota_wypłaty_PLN
Wiktor	Wilk	15000.0
Lucyna	Lis	20000.0

Złączenia

Przykład 6: Wyszukać nazwy tych działów, których pracownicy realizowali zadanie „Projekt symulatora”



Złączenia

```
SELECT d.nazwa  
FROM Dział d  
JOIN Pracownik USING (id_działu)  
JOIN PracownikZespołu USING (id_pracownika)  
JOIN Zespół z USING (id_zespołu)  
WHERE z.nazwa_zadania = 'Projekt symulatora';
```

nazwa
Dział Wdrożeń
Dział Wdrożeń
Dział Analiz

Złączenia

Klauzula `SELECT DISTINCT` zwraca tylko niepowtarzające się wartości

Gdy używamy `SELECT DISTINCT`, system zarządzania bazą danych usuwa duplikaty, pozostawiając po jednym wierszu dla każdego zestawu danych

Złączenia

```
SELECT DISTINCT d.nazwa  
FROM Dział d  
JOIN Pracownik USING (id_działu)  
JOIN PracownikZespołu USING (id_pracownika)  
JOIN Zespół z USING (id_zespołu)  
WHERE z.nazwa_zadania = 'Projekt symulatora';
```

nazwa
Dział Wdrożeń
Dział Analiz

Złączenia

Przykład 7: Wyszukać imiona i nazwiska tych pracowników, którzy pracują w tym samym dziale, co Olga

E Pracownik p1

id_pracownika INT [KG]

id_działu INT [KO]

imię TEXT

nazwisko TEXT

E Pracownik p2

id_pracownika INT [KG]

id_działu INT [KO]

imię TEXT

nazwisko TEXT

Złączenia

```
SELECT p1.imię, p1.nazwisko  
FROM Pracownik p1  
JOIN Pracownik p2 USING (id_działu)  
WHERE p2.imię = 'Olga';
```

imię	nazwisko
Olga Wiktor	Owca Wilk

Złączenia

Przykład 8: Wyszukać imiona i nazwiska tych pracowników, którzy pracują w tym samym dziale, co Olga, oprócz Olgi

E Pracownik p1

id_pracownika INT [KG]

id_działu INT [KO]

imię TEXT

nazwisko TEXT

E Pracownik p2

id_pracownika INT [KG]

id_działu INT [KO]

imię TEXT

nazwisko TEXT

Złączenia

```
SELECT p1.imię, p1.nazwisko  
FROM Pracownik p1  
JOIN Pracownik p2 USING (id_działu)  
WHERE p2.imię = 'Olga'  
AND p1.imię <> 'Olga';
```

imię	nazwisko
Wiktor	Wilk

Złączenia

Przykład 9: Wyszukać imiona i nazwiska tych pracowników, którzy otrzymali wypłaty w ramach tych zadań, którymi kierował Wiktor

E Pracownik p1

id_pracownika INT [KG]

id_działu INT [KO]
imię TEXT
nazwisko TEXT

E Zespół

id_zespołu INT [KG]

id_pracownika_kierownika_zespołu [KO]
nazwa_zadania TEXT

E PracownikZespołu

id_pracownika_zespołu INT [KG]

id_pracownika INT [KO]
id_zespołu INT [KO]
kwota_wypłaty_PLN REAL

E Pracownik p2

id_pracownika INT [KG]

id_działu INT [KO]
imię TEXT
nazwisko TEXT

kierownik zespołu

Złączenia

```
SELECT p1.imię, p1.nazwisko  
FROM Pracownik p1  
JOIN PracownikZespołu USING (id_pracownika)  
JOIN Zespół z USING (id_zespołu)  
JOIN Pracownik p2 ON z.id_pracownika_kierownika_zespołu = p2.id_pracownika  
WHERE p2.imię = 'Wiktor';
```

imię	nazwisko
Wiktor	Wilk
Olga	Owca
Olga	Owca
Wiktor	Wilk
Kasia	Kot

Złączenia

```
SELECT DISTINCT p1.imię, p1.nazwisko  
FROM Pracownik p1  
JOIN PracownikZespołu USING (id_pracownika)  
JOIN Zespół z USING (id_zespołu)  
JOIN Pracownik p2 ON z.id_pracownika_kierownika_zespołu = p2.id_pracownika  
WHERE p2.imię = 'Wiktor';
```

imię	nazwisko
Wiktor	Wilk
Olga	Owca
Kasia	Kot

To już koniec :-)

Tematy dzisiejszego wykładu

Z tego wykładu dowiedziecie się:

- + o klauzuli WHERE

- + o tym, jak zmieniać bazę danych instrukcją
ALTER TABLE i UPDATE

- + o tym, jak zadawać zapytania
ze złączeniami, czyli z klauzulą JOIN

Proszę o pytania :-)

Zapraszam was na następny wykład. Dowiedcie się z niego:

- + jak operator `||` łączy teksty
- + jak działa podwójny cudzysłów `"`
- + jak działają funkcje agregujące
- + jaka jest kolejność klauzul w zapytaniu
- + jak działa klauzula `GROUP BY`
- + jak działa klauzula `HAVING`
- + jak system zarządzania bazą danych wykonuje zapytanie

Podziękowania

Podziękowania —<—+—@

Dziękuję:

+ prof. Stanisławowi Koziełskiemu